

SISTEM ANTRIAN RUMAH SAKIT

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Muhammad Daffadhiya Wirawan J0403251007

Ibrahim Ahmad Idhofi J0403251107

Muhammad Ramdhan Maulana J0403251110

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan proyek akhir mata kuliah Algoritma & Struktur Data (TPL2106) dengan judul "Sistem Antrian Rumah Sakit" ini tepat pada waktunya untuk memenuhi tugas akhir *Project-Based Learning* (PBL). Kami menyampaikan terima kasih yang sebesar-besarnya kepada dosen pengampu mata kuliah, Ibu Dr. Shelve Nidya Neyman, S.Kom., M.Si. dan Ibu Dr. Sofiyanti Indriasari, S.Kom., M.Kom., atas bimbingan dan arahan yang diberikan selama proses pembelajaran, serta kepada seluruh anggota kelompok atas kerja sama dan dedikasi penuh dalam pengembangan aplikasi ini. Kami menyadari bahwa laporan ini masih memiliki keterbatasan, sehingga kritik dan saran yang membangun sangat kami harapkan demi penyempurnaan di masa mendatang, dan kami berharap laporan ini dapat memberikan manfaat serta wawasan bagi pembaca terkait implementasi struktur data dan algoritma dalam memecahkan permasalahan nyata di bidang teknologi rekayasa perangkat lunak.

Bogor, 24 Juni 2026

DAFTAR ISI

BAB I. PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan Proyek.....	4
1.3 Ruang Lingkup.....	4
BAB II. ANALISIS DAN PERANCANGAN.....	6
2.1 Deskripsi Sistem.....	6
2.2 Struktur Data yang Digunakan.....	6i
2.3 Algoritma yang Digunakan.....	8
2.4 Diagram Alir Program (Flowchart).....	9
2.5 Struktur Program.....	11
BAB III. IMPLEMENTASI PROGRAM.....	13
3.1 Tampilan Menu Utama.....	13
3.2 Fitur Create.....	13
3.3 Fitur Read.....	14
3.4 Fitur Update.....	14
3.5 Fitur Delete.....	15
3.6 Fitur Searching.....	15
3.7 Fitur Sorting.....	16
3.8 Penyimpanan Data (File Handling).....	16
3.9 Validasi Input dan Error Handling.....	16
BAB IV. PENGUJIAN DAN ANALISIS.....	18
4.1 Skenario Pengujian.....	18
4.2 Kelebihan Sistem.....	19
4.3 Keterbatasan Sistem.....	19
BAB V. KENDALA DAN SOLUSI.....	20
BAB VI. PEMBAGIAN TUGAS ANGGOTA.....	21
Berikut adalah pembagian tugas dari kelompok kami.....	21
BAB VII. KESIMPULAN DAN SARAN.....	22
LAMPIRAN.....	24

BAB I. PENDAHULUAN

1.1 Latar Belakang

Pengelolaan antrean merupakan aspek krusial dalam operasional sebuah instansi pelayanan kesehatan untuk memastikan efisiensi dan kenyamanan pasien. Saat ini, sistem antrean yang dilakukan secara manual sering kali menimbulkan ketidakteraturan, sehingga diperlukan sistem digitalisasi yang mampu mengelola data pasien secara lebih terstruktur. Proyek ini dikembangkan sebagai solusi untuk menjawab permasalahan tersebut melalui aplikasi berbasis terminal yang menerapkan konsep *Queue* (antrean) dengan sistem prioritas. Selain mengimplementasikan struktur data *Queue* untuk mengatur urutan pasien berdasarkan prinsip *First In First Out* (FIFO) dan kondisi darurat, sistem ini juga memanfaatkan fitur *file handling* untuk memastikan data pasien tersimpan secara permanen ke dalam format CSV. Dengan demikian, aplikasi ini diharapkan mampu membantu staf rumah sakit dalam mengelola antrean dengan lebih akurat, transparan, dan efisien.

1.2 Tujuan Proyek

Adapun tujuan dari pelaksanaan proyek ini adalah sebagai berikut:

1. Mengembangkan aplikasi sistem antrean berbasis *Command Line Interface* (CLI) menggunakan bahasa pemrograman Python.
2. Mengimplementasikan konsep struktur data *Queue* dan algoritma pendukung, seperti *searching* dan *sorting*, untuk mengoptimalkan pengelolaan data pasien.
3. Menerapkan mekanisme *file handling* agar data antrean dapat tersimpan secara permanen dan dapat diakses kembali saat program dijalankan ulang.
4. Memberikan solusi berbasis teknologi terhadap permasalahan nyata terkait pengelolaan antrean yang tidak terstruktur di lingkungan rumah sakit.

1.3 Ruang Lingkup

1. Platform CLI (Console)
2. Bahasa Python
3. Penyimpanan file CSV/TXT
4. Fitur CRUD
5. Searching & Sorting

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem Antrian Rumah Sakit ini merupakan aplikasi berbasis *Command Line Interface* (CLI) yang dirancang untuk mereplikasi alur manajemen antrean pasien di lingkungan fasilitas kesehatan. Aplikasi ini berfungsi sebagai pusat pengelolaan data pasien, di mana pengguna (administrator) dapat melakukan pendaftaran pasien baru, memanggil pasien berikutnya sesuai urutan, serta memantau daftar antrean aktif secara *real-time*.

Salah satu fitur utama sistem ini adalah penerapan algoritma antrean dengan sistem prioritas, yang memungkinkan pasien dalam kategori darurat untuk dilayani lebih awal dibandingkan pasien dengan kategori normal, guna mencerminkan situasi nyata di rumah sakit. Untuk menjamin persistensi data, sistem dilengkapi dengan fitur *file handling* yang secara otomatis menyimpan kondisi antrean ke dalam file `antrean.csv` setiap kali program ditutup atau terjadi perubahan data. Dengan demikian, data pasien akan tetap tersedia dan dapat dimuat kembali saat aplikasi dijalankan ulang, sehingga meminimalisir risiko kehilangan data dan meningkatkan efisiensi operasional sistem. Selain itu, sistem menyediakan fitur tambahan berupa *searching* dan *sorting* yang memudahkan administrator dalam mengelola serta mencari data pasien secara spesifik tanpa mengubah urutan antrean utama.

2.2 Struktur Data yang Digunakan

Dalam pengembangan aplikasi Sistem Antrian Rumah Sakit, kami mengimplementasikan beberapa struktur data untuk memastikan sistem berjalan efisien dan sesuai dengan alur kerja pelayanan pasien. Berikut adalah rincian struktur data yang digunakan:

Struktur Data	Fungsi
Queue	Mengelola antrean utama pasien berdasarkan prinsip First In First Out (FIFO) serta sistem prioritas.

Dictionary	Menyimpan data pasien secara terstruktur, mempermudah akses atribut seperti nama, kategori, dan ID pasien.
List	Menyimpan kumpulan data pasien dalam antrean aktif untuk kebutuhan viewing, searching, dan sorting.

2.3 Alasan Pemilihan Struktur Data:

1. **Queue:** Struktur data ini dipilih karena sangat sesuai dengan prinsip antrean rumah sakit, di mana pasien yang datang lebih dahulu akan dilayani terlebih dahulu (*First In First Out*). Selain itu, kami memodifikasi implementasi *Queue* untuk mendukung sistem prioritas, sehingga pasien dengan kategori darurat dapat diproses mendahului antrean normal guna menangani situasi gawat darurat secara nyata.
2. **Dictionary:** Kami menggunakan *Dictionary* untuk merepresentasikan objek pasien. Hal ini memberikan fleksibilitas tinggi dalam memanipulasi data pasien, karena setiap atribut (nama, kategori, ID) dapat diakses dengan kunci (*key*) yang unik, sehingga mempermudah proses manipulasi data di dalam program.
3. **List:** Penggunaan *List* bertujuan untuk menyimpan daftar antrean aktif di memori saat program berjalan. Struktur ini dipilih karena kemudahannya dalam proses iterasi, yang sangat diperlukan ketika kami melakukan fitur pencarian (*searching*) atau pengurutan (*sorting*) untuk menampilkan daftar antrean kepada pengguna tanpa mengubah urutan antrean utama yang tersimpan di dalam sistem *Queue*.

2.4 Algoritma yang Digunakan

Dalam proyek "Sistem Antrian Rumah Sakit" ini, kami menerapkan beberapa algoritma untuk memastikan efisiensi pencarian data, pengurutan antrean, serta manajemen prioritas pasien. Berikut adalah rincian algoritma yang digunakan

1. Searching (Pencarian)

Algoritma: *Sequential Search* (Pencarian Linear).

Alasan Penggunaan: Kami menggunakan *Sequential Search* karena data pasien disimpan dalam struktur *Linked List*. Algoritma ini menelusuri setiap *node* mulai dari *head* hingga *tail* untuk mencocokkan kriteria pencarian (ID, nama, atau waktu). Metode ini sangat efektif dan sederhana untuk struktur data *Linked List* yang tidak mendukung akses acak (*random access*), serta memungkinkan pencarian *partial match* (pencarian sebagian nama) secara fleksibel.

2. Sorting (Pengurutan)

Algoritma: *Merge Sort*.

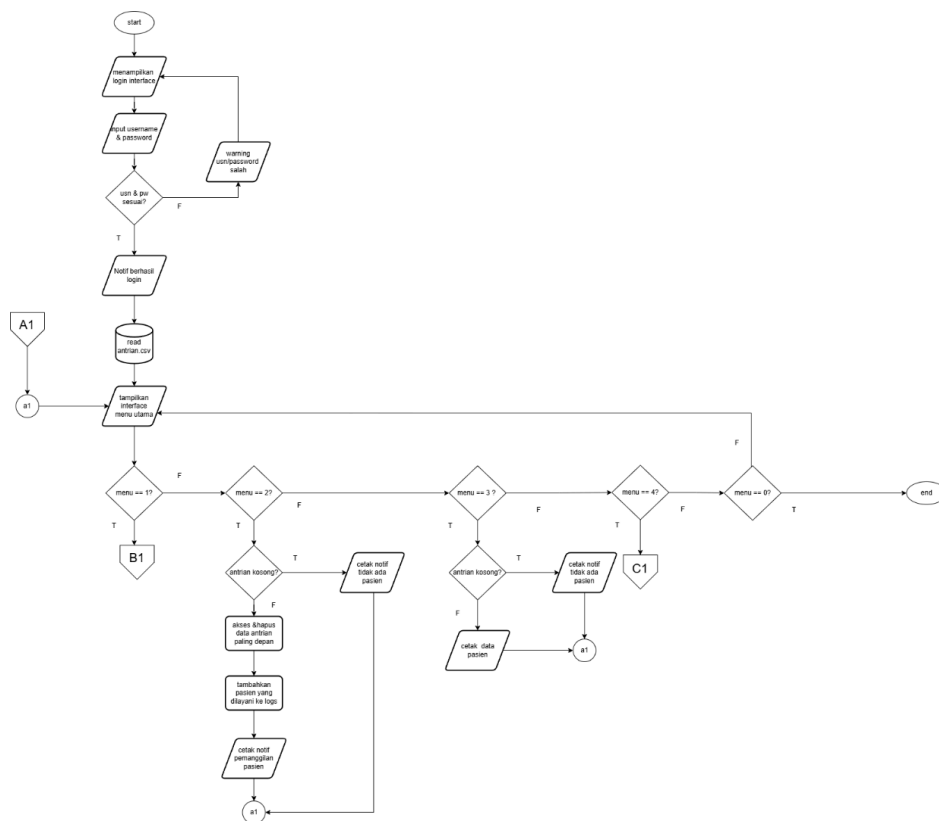
Alasan Penggunaan: *Merge Sort* dipilih karena efisiensinya yang stabil dengan kompleksitas waktu $O(n \log n)$. Algoritma ini digunakan untuk mengurutkan daftar pasien berdasarkan nama (alfabetis) maupun kategori tanpa mengubah struktur *Linked List* antrian utama yang asli. Dengan pendekatan *divide and conquer*, *Merge Sort* mampu menangani data dalam jumlah yang lebih besar dengan performa yang tetap konsisten.

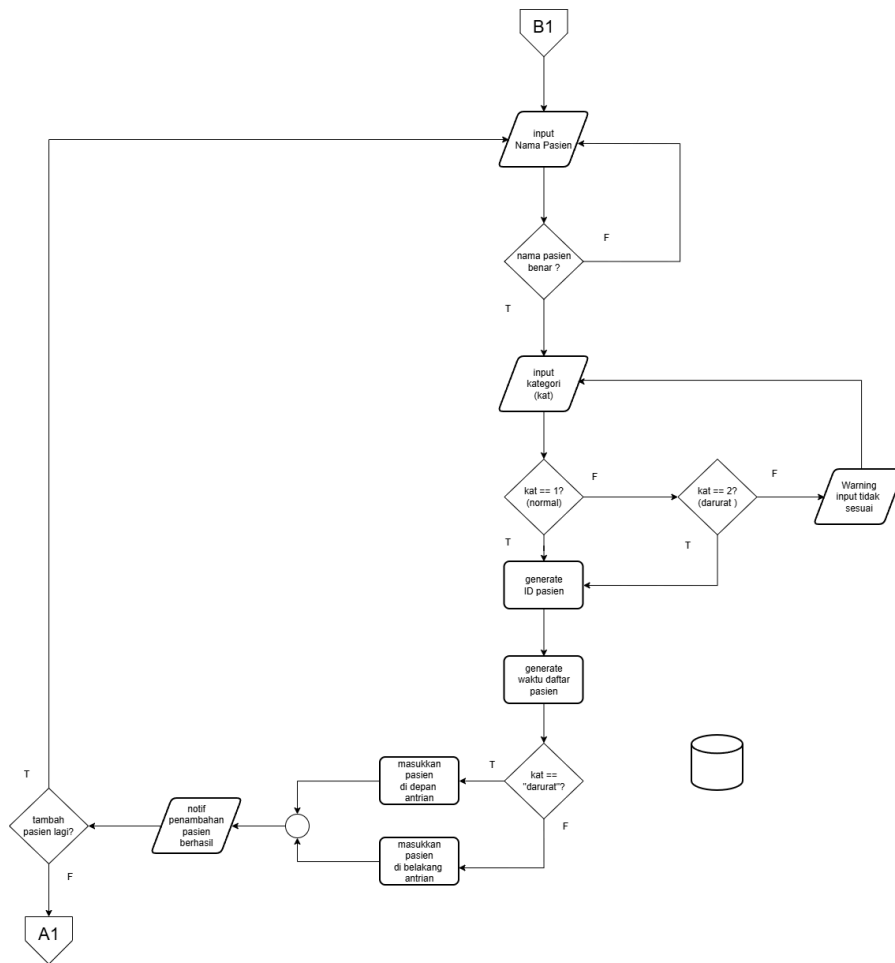
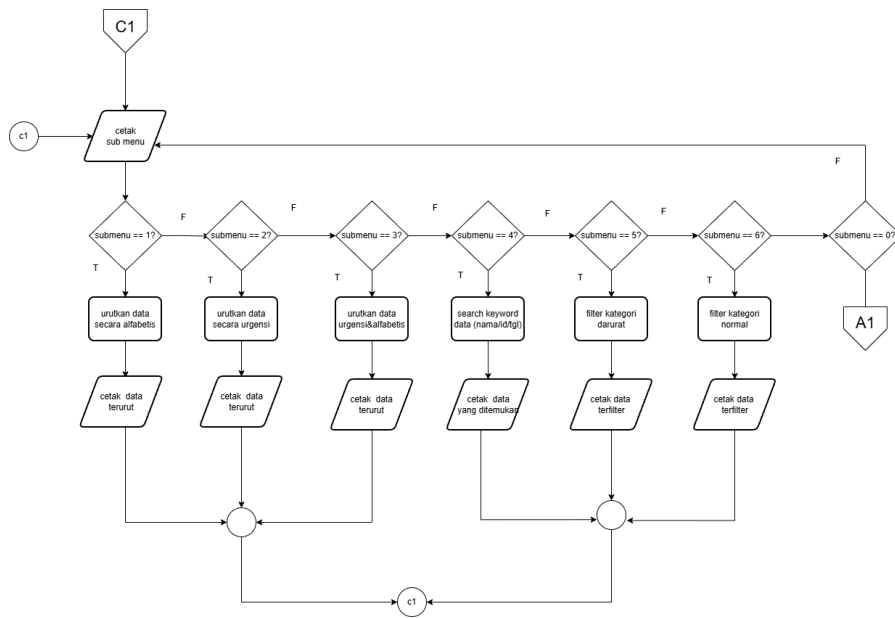
3. Priority Queue (Manajemen Prioritas)

Algoritma: *Priority Queue Logic* (FIFO-Priority Hybrid).

Alasan Penggunaan: Kami menerapkan logika antrian khusus di mana pasien dengan kategori "Darurat" akan dimasukkan ke dalam antrian setelah pasien darurat terakhir, namun tetap berada di depan seluruh pasien "Normal". Hal ini memastikan bahwa prinsip *First In First Out* (FIFO) tetap terjaga di antara pasien dalam kategori yang sama, sementara tingkat urgensi medis tetap menjadi prioritas utama pelayanan.

2.5 Diagram Alir Program (Flowchart)





2.6 Struktur Program

Aplikasi "Sistem Antrian Rumah Sakit" dikembangkan dengan arsitektur modular yang memisahkan logika bisnis, antarmuka pengguna, dan lapisan persistensi data. Pendekatan ini bertujuan untuk meningkatkan *maintainability* (kemudahan pemeliharaan) dan *readability* (keterbacaan kode). Berikut adalah rincian teknis dari komponen-komponen penyusun sistem:

1. **main.py**: Berperan sebagai pintu masuk utama aplikasi. Berkas ini menginisialisasi sistem, memuat data dari *file* ke dalam memori, serta mengelola *looping* utama menu sehingga aplikasi tetap berjalan hingga pengguna memilih untuk keluar.
2. **engine.py**: Modul ini menampung implementasi struktur data *Queue* yang dimodifikasi. Di sini terdapat fungsi-fungsi krusial seperti:
 - a. Logika *Priority Queue* untuk menyisipkan pasien darurat ke posisi depan tanpa merusak urutan pasien darurat lainnya.
 - b. Algoritma *Merge Sort* yang digunakan untuk pengurutan data saat *viewing*, yang membagi antrian menjadi sub-list lebih kecil agar efisiensi pengurutan mencapai $O(n \log n)$.
 - c. Fungsi *Sequential Search* untuk memfilter data pasien berdasarkan ID atau nama secara linear.
3. **interface.py**: Berfungsi sebagai *interface* pengguna. Modul ini bertanggung jawab menangani input dari pengguna, melakukan validasi tipe data (misalnya memastikan input usia atau kategori berupa integer), serta memformat *output* daftar antrian ke dalam bentuk tabel yang rapi menggunakan teknik *string formatting* agar informasi mudah dibaca.
4. **storage.py**: Modul ini menangani operasi I/O (*Input/Output*). Ia menyediakan fungsi untuk membaca data dari berkas CSV menjadi *list of dictionaries* saat program dimulai, serta menuliskan ulang isi memori ke dalam berkas saat program dihentikan (*save on exit*), guna memastikan persistensi data.

5. **antrean.csv**: Berkas penyimpan kondisi antrean saat ini. Format CSV dipilih karena strukturnya yang ringan, mudah dibaca oleh program maupun manusia, serta kompatibel dengan aplikasi *spreadsheet* lainnya.
6. **log_pelayanan.csv**: Berkas yang berfungsi sebagai jurnal/riwayat. Setiap pasien yang telah dipanggil (keluar dari antrean) akan direkam datanya ke sini untuk kebutuhan audit atau analisis pelayanan di masa mendatang.

Struktur ini memastikan bahwa jika terjadi kendala pada tampilan (misalnya ingin mengubah warna *interface*), kami hanya perlu menyentuh berkas `interface.py` tanpa mengganggu logika antrean yang sensitif di `engine.py`.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

```
=====
                        SISTEM ANTRIAN KLINIK
                Pelayanan Kesehatan Cepat & Responsif
=====
STATS: [Menunggu: 17] | [Pasien dilayani]: 0]
-----
[1] Registrasi Pasien Baru
[2] Panggil Pasien Berikutnya
[3] Lihat Antrian Aktif (Fix Order)
[4] Pencarian & Pengurutan Pasien
[0] Keluar & Simpan Data
-----
Pilih menu [0-4]: █
```

3.2 Fitur Create

```
=== REGISTRASI PASIEN BARU ===

Nama Pasien : Idhofi

[VERIFIKASI ADMIN]
Apakah nama 'Idhofi' sudah benar dan sesuai KTP/BPJS?
Konfirmasi [Y/N]: y

Kategori Kondisi Pasien:
[1] Normal
[2] Darurat (Prioritas)
Pilih [1/2] : 1

✅ Pasien P021 berhasil terdaftar!

-----
Apakah ingin mendaftarkan pasien lain?
Input lagi? [Y/N]: █
```

Fitur create atau dalam program kami fitur **registrasi pasien baru**, berfungsi untuk untuk menambah antrean pasien baru. User memberikan input **nama**, lalu dikonfirmasi kebenaran datanya terlebih dahulu. Setelah itu, user memberikan input

angka 1 atau 2 untuk kategori kondisi pasien. Setelah antrian terdaftar, program akan menawarkan untuk mendaftarkan pasien lagi agar **user tidak perlu repot keluar masuk menu**.

3.3 Fitur Read



ID	NAMA PASIEN	KATEGORI
P011	Pitching	Darurat
P017	Hiba	Darurat
P019	Haji Dapd=a	Darurat
P001	Ramd	Normal
P002	Ram	Normal
P003	Ramdhan	Normal
P004	Ibrahim	Normal
P005	Dapdih	Normal
P008	Arka	Normal
P009	James	Normal
P012	Ibrahim	Normal
P013	Ibradih	Normal
P014	ibra	Normal
P015	Ibramul	Normal
P016	Ghidorah	Normal
P018	Haji Abim	Normal
P020	gh	Normal
P021	Idhofi	Normal

Tekan Enter untuk kembali...

Fitur **lihat daftar antrian** merupakan implementasi dari fitur read. Di menu ini, user bisa melihat daftar antrian dalam tabel mulai dari **ID, Nama Pasien, dan Kategori**. Daftar dibuat dalam bentuk tabel dan kategori diberikan warna agar lebih mudah untuk dibaca.

3.4 Fitur Update

Berdasarkan hasil diskusi kelompok, kami memutuskan untuk tidak menggunakan fitur update karena kami melihat bahwa fitur ini tidak relevan. Mengingat data antrian program ini hanya berisi **ID, Nama Pasien, Kategori dan Tanggal**, maka menurut kami tidak ada urgensi untuk mengubah salah satu dari data-data tersebut. Sebagai gantinya, kami memberikan konfirmasi lanjutan ketika user ingin mendaftarkan antrian pasien baru.

3.5 Fitur Delete

```
PASIENT HARAP MENUJU RUANG DOKTER

ID PASIEN : P011
NAMA      : PITCHING
STATUS    : DARURAT
⚠️PERINGATAN: SEGERA BERIKAN TINDAKAN MEDIS!

Tekan [Enter] setelah pasien selesai dilayani...
```

Kami mengimplementasikan fitur delete menjadi fitur **layani antrean pasien**. Karena data sudah diurut berdasarkan **priority queue**, maka data yang pertama dilayani adalah data yang **berkategori darurat dan lebih dulu masuk**.

3.6 Fitur Searching

```
=====
PENCARIAN, PENGURUTAN & FILTER ANTRIAN
Fitur Analisis dan Filtrasi Data Pasien
=====

— Opsi Pengurutan (Sorting) —
[1] Urut Alfabetis      (A-Z)
[2] Urut Urgensi        (Darurat → Normal)
[3] Urut Gabungan       (Darurat A-Z → Normal A-Z)

— Opsi Pencarian (Searching) —
[4] Cari Pasien         (Nama / ID / Tanggal)

— Opsi Penyaringan (Filtering) —
[5] Tampilkan Khusus Pasien Darurat
[6] Tampilkan Khusus Pasien Normal

[0] Kembali ke Menu Utama

Pilih menu [0-6]:
```

Fitur search pada program ini masih menggunakan **algoritma linear search** karena struktur data yang kami gunakan adalah linked list. Namun, fitur ini dinilai sukses karena berhasil menemukan data berdasarkan keyword **Nama**, **ID**, atau **Tanggal**. Jika keyword yang diberikan user tidak spesifik, program tetap bisa memberikan list antrean tanpa memberikan error.

```
=== PENCARIAN DATA PASIEN ===

Masukkan Nama / ID / Tanggal Pasien: P001

Mencari pasien dengan keyword 'P001'...
✅ DITEMUKAN: P001 - Ramd [Normal] (Daftar: 2026-05-19 21:48:58)

Tekan Enter untuk kembali...
```

3.7 Fitur Sorting

Algoritma yang kami gunakan untuk fitur sorting ini adalah **algoritma merge sort**. Secara default, antrean sudah diurutkan berdasarkan prinsip **Priority Queue**, artinya data diurutkan berdasarkan kategori terlebih dahulu, lalu diurutkan sesuai dengan prinsip FIFO. Akan tetapi, kami tetap menyediakan opsi tambahan seperti diurut berdasarkan abjad, juga diurut berdasarkan kategori lalu diurut sesuai abjad. Selain itu, kami juga memberikan opsi **filtering** untuk menampilkan antrean **hanya pada kategori tertentu**.

```
=== ANTRIAN ALFABETIS (A-Z) ===
```

ANTRIAN - URUT ALFABETIS (A-Z)				
No	ID	NAMA PASIEN	STATUS	WAKTU DAFTAR
1	P008	Arka	Normal	2026-05-20 05:50:17
2	P005	Dapdih	Normal	2026-05-20 05:47:39
3	P020	gh	Normal	2026-06-22 21:36:58
4	P016	Ghidorah	Normal	2026-05-20 10:37:16
5	P018	Haji Abim	Normal	2026-05-20 15:12:02
6	P019	Haji Dapd-a	Darurat	2026-05-20 15:12:49
7	P017	Hiba	Darurat	2026-05-20 10:37:43
8	P014	ibra	Normal	2026-05-20 05:51:23
9	P013	Ibradih	Normal	2026-05-20 05:51:12
10	P004	Ibrahim	Normal	2026-05-20 05:47:24
11	P012	Ibrahim	Normal	2026-05-20 05:51:06
12	P015	Ibramul	Normal	2026-05-20 05:51:37
13	P021	Idhofi	Normal	2026-06-26 19:48:47
14	P009	James	Normal	2026-05-20 05:50:25
15	P002	Ram	Normal	2026-05-20 05:23:01
16	P001	Ramd	Normal	2026-05-19 21:48:58
17	P003	Ramdhan	Normal	2026-05-20 05:47:16

Total: 17 pasien

Tekan Enter untuk kembali...

3.8 Penyimpanan Data (File Handling)

Kami menggunakan file CSV karena jika dibandingkan dengan menggunakan file TXT, menggunakan CSV bisa mengefisiensi algoritma untuk memisahkan data karena CSV Reader sudah otomatis memisahkan data berdasarkan tanda koma.

File dibaca menggunakan fungsi **with open() as file** lalu data dipisah secara terurut ke atribut **nama**, **kategori**, **ID**, dan **waktu**. Ketika program sudah selesai, program akan otomatis menyimpan data kembali ke CSV dengan menyimpan data satu per satu secara linear.

3.9 Validasi Input dan Error Handling

Kesalahan Input	Penanganan
-----------------	------------

Input selain dari opsi input yang disediakan	Program meminta input ulang
Gagal menyimpan file	Menampilkan pesan log
File belum tersedia	Program membuat file baru

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Deskripsi	Hasil
1	Otentikasi Login Admin	Memberikan sensor karakter dengan (*)	Berhasil
2	Registrasi Pasien	Mendaftarkan pasien dengan validasi input dan konfirmasi kesesuaian data	Berhasil
3	Pengujian Priority Queue	Mendaftarkan dan memastikan pasien dengan kategori “darurat” terdaftar di depan pasien “normal” dan sesuai dengan prinsip FIFO	Berhasil
4	Pemanggilan Pasien	Memanggil pasien dari antrean pertama	Berhasil
5	ID Generator	Menguji ID yang dihasilkan tidak berubah setelah didaftarkan ke dalam antrean	Berhasil
6	Show Data	Menampilkan seluruh data yang sudah terurut	Berhasil
7	Sorting Data	Mengurutkan data berdasarkan abjad, priority, dan gabungan keduanya	Berhasil
8	Searching Data	Melakukan pencarian sebagian (partial match) berdasarkan Nama, ID, dan Tanggal	Berhasil
9	File Handling	Membaca dan menyimpan data ke dalam file antrean.csv	Berhasil

4.2 Kelebihan Sistem

1. Program sudah memiliki Error Handling yang baik di mana setiap inputan sudah dikontrol agar tetap sesuai dengan alur program.
2. Program sudah otomatis membaca ketika memulai program dan otomatis menyimpan ketika program selesai.
3. Efisiensi algoritma pengurutan menggunakan algoritma **merge sort** dengan time complexity $O(n \log n)$.
4. Interface yang mudah dibaca karena output diberikan visual berupa warna yang kontras dan gap yang pas antarteksnya.

4.3 Keterbatasan Sistem

1. Masih berbasis CLI atau hanya bisa dioperasikan melalui terminal.
2. Masih menggunakan Linear Search
3. Belum mendukung multiuser
4. Belum mendukung database

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Data ID pasien ter-reset (bentrok) saat program di-restart	Mencari referensi id generator dari program yang serupa di internet
Input kategori tidak valid (selain 1 atau 2) dianggap sebagai kategori darurat	Menambahkan <i>Error Handling</i> (looping input hingga sesuai)
Fitur <i>sorting</i> dasar yang kurang memenuhi kebutuhan tampilan	Menambahkan fitur <i>sorting</i> untuk <i>viewing</i> (tanpa mengubah urutan data asli)
Interface monoton dengan teks satu warna (putih)	Menambahkan warna sebagai pesan psikologi (<i>red</i> untuk peringatan, <i>green</i> untuk berhasil)
Masalah <i>bug</i> dan <i>error</i> pada versi awal program	Melakukan <i>monitoring</i> dan perbaikan berkelanjutan
Rencana fitur riwayat antrean yang kurang relevan	Konsultasi dengan asdos dan membatalkan fitur yang tidak sesuai business case

Proyek ini menjadi proses pembelajaran yang interaktif bagi kami dalam menyeimbangkan antara idealisme teknis dan kebutuhan nyata di lapangan. Melalui fase *monitoring*, kami tidak hanya dipaksa untuk berpikir logis dalam mengamankan integritas data dan memvalidasi setiap input pengguna, tetapi juga diajak untuk melihat program dari sudut pandang psikologis dan estetika melalui pemanfaatan warna pada antarmuka. Tantangan terbesar hadir saat kami harus belajar menahan diri dan bersikap adaptif; baik ketika menerima arahan asisten dosen untuk mengembangkan fitur visualisasi data yang lebih dinamis, maupun saat kami harus merelakan fitur riwayat antrean demi menjaga fokus aplikasi agar tetap efisien dan relevan dengan kebutuhan operasional seorang admin.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Berikut adalah pembagian tugas dari kelompok kami.

Nama	Tugas Utama	Rincian Detail Tugas
Muhammad Daffadhiya Wirawan	File Handling, Searching, Sorting	1. Menghubung Program ke media penyimpanan CSV lewat fungsi load dan save. 2. Membuat algoritma Merge Sort. 3. Membuat penelusuran data Linear Search.
Ibrahim Ahmad Idhofi	Alur Utama Program, Interface, Pengujian & error handling	1. Membangun fitur alur penanganan input menu utama. 2. Melakukan pengujian validasi input (<i>Foolproof System</i>) agar program tetap berjalan walau terdapat kesalahan user. 3. Membuat interface yang terstruktur & memanfaatkan psikologi warna agar program dapat digunakan dengan mudah.
Muhammad Ramdhan Maulana	Struktur data utama & crud	1. Merancang arsitektur model simpul class Node dan instansiasi class PriorityQueue. 2. Menyusun logika penyisipan prioritas enqueue dequeue, dan fungsi lain yang diperlukan. 3. Mengimplementasikan sistem keamanan gerbang login admin

BAB VII. KESIMPULAN DAN SARAN

8.1 Kesimpulan

Walau belum dapat diimplementasikan pada kasus di dunia nyata, program kami tetap berhasil dalam merancang sistem antrian klinik yang efisien. Menggunakan struktur data *Priority Queue* berbasis *linked list* program ini dapat mengunci fungsi pendaftaran dan pemanggilan pasien dalam kecepatan konstan $O(1)$. Struktur dinamis tersebut sangat efisien dalam memanipulasi alokasi memori RAM secara langsung. Sehingga sistem dapat secara otomatis menyortir pasien berdasarkan tingkat kedaruratannya menggunakan prinsip FIFO khusus demi menjaga keadilan respons pelayanan kesehatan.

Di samping itu, keberhasilan proyek ini didukung oleh integrasi algoritma pengurutan **Merge Sort** yang memberikan garansi kecepatan waktu $O(n \log n)$, serta keberhasilan implementasi modul *File Handling* berbasis CSV (*antrean.csv* dan *log_pelayanan.csv*). Kolaborasi ini memberikan pengalaman berharga mengenai pentingnya stabilitas data agar urutan kedatangan asli pasien berabjad sama tidak teracak, sekaligus menjamin keamanan data secara permanen saat program dimatikan maupun dinyalakan ulang. Pengalaman krusial lainnya diperoleh melalui penerapan metode *Defensive Programming*, di mana pembuatan gerbang otentikasi login admin bersensor bintang serta validasi input menu sukses mengamankan sistem secara menyeluruh dari risiko kelalaian manusia petugas loket.

8.2 Saran

Berikut adalah beberapa fungsionalitas sistem pada masa mendatang, berikut adalah beberapa fitur yang dapat dikembangkan.

1. Fitur Integrasi Basis Data

Penyimpanan data menggunakan csv memiliki banyak keterbatasan. Penggunaan DBMS yang terstruktur dapat meningkatkan skalabilitas program.

2. Fitur Sistem Estimasi Waktu Tunggu Dinamis

Mengembangkan algoritma untuk menghitung perkiraan waktu tunggu secara riil berdasarkan jumlah antrian aktif dan rata-rata durasi penanganan pasien oleh dokter

sebelumnya. Informasi waktu ini dapat ditampilkan langsung pada antrean aktif untuk meningkatkan kenyamanan pasien.

3. **Fitur *Auto-Reset* Statistik Harian**

Menambahkan logika pengecekan tanggal otomatis agar statistik dashboard pendaftaran kembali ke angka nol setiap hari baru tanpa memerlukan penghapusan berkas CSV secara manual.

LAMPIRAN

A. Link Github

https://github.com/Ram24-L/ASD_KEL1_TPLB.git

B. Screenshot Program

```
=====
                        SISTEM ANTRIAN KLINIK
                    Pelayanan Kesehatan Cepat & Responsif
=====
STATS: [Menunggu: 16] | [Pasien dilayani]: 0]
=====
[1] Registrasi Pasien Baru
[2] Panggil Pasien Berikutnya
[3] Lihat Antrian Aktif (Fix Order)
[4] Pencarian & Pengurutan Pasien
[0] Keluar & Simpan Data
=====
Pilih menu [0-4]: █
```

C. Struktur Folder Proyek

Proyek/

```
|— main.py
|— interface.py
|— storage.py
|— engine.py
|— antrean.csv
└— log_pelayanan.csv
```


D. Commit History Github

Commits on May 18, 2026
post monitoring code Ram24-L committed on May 18 06cf1f9
Commits on May 13, 2026
jika Ram24-L committed on May 13 165a224
version 1 Ram24-L committed on May 13 a6c2636
updating storage.py Dapdh committed on May 13 c55e183
pushing storage Dapdh committed on May 13 a10619f
Commits on Apr 27, 2026
base engine.py Ram24-L committed on Apr 28 3dbd42e
Merge branch 'master' Ram24-L committed on Apr 28 543cc48
core file initialization Ram24-L committed on Apr 28 12db8a6
Commits on Feb 18, 2026
Add initial content to readme Ram24-L authored on Feb 18 Verified 07c3f0c

Commits on May 19, 2026
interface Ram24-L committed on May 20 61da7b9
added login feature & debugging Ram24-L committed on May 20 d497f1f
interface consistency & log view removal Ram24-L committed on May 20 82fab63
code review Ram24-L committed on May 19 634f6ba
merge conflict main Ram24-L committed on May 19 5b1063b
Merging feature & debugs Ram24-L committed on May 19 6eb0ba2
Merge branch 'Debug-phase' Dapdh committed on May 19 072a284
committing before merge Dapdh committed on May 19 185ec94
Merge pull request #2 from Ram24-L/interface ibrdh authored on May 19 Verified d7b07be
Merge branch 'main' into interface ibrdh authored on May 19 Verified 2f22380
integrating search and sort Dapdh committed on May 19 37b2f39
Mengubah UI terminal jadul menjadi tabel yang colorful ibrdh committed on May 19 071684e
Creating New Branch Dapdh committed on May 19 6975dfe
new attempt Dapdh committed on May 19 efc236b
Penambahan atribut Waktu dengan format DD/MM/YY HH/MM/SS di timezone GMT +7 ibrdh committed on May 19 0bdb9d1
Fix algoritma id generator dan fix urutan antrian pada pasien darurat ibrdh committed on May 19 e9ccb63

Commits		
main		
All users All time		
Commits on Jun 22, 2026		
perapihan	Ram24-L committed 4 days ago	e6efdc0
Merge branch 'main' of https://github.com/Ram24-L/ASD_KEL1_TPLB	Ram24-L committed 4 days ago	7f91d4f
final ?	Ram24-L committed 4 days ago	9c6c217
Commits on May 20, 2026		
revisi	Dapdih committed on May 20	d18d61f
new revise	Dapdih committed on May 20	6a4f891
revise the sorting	Dapdih committed on May 20	06a1339
Perapihan interface dan keterbacaan program	lbrhdhf committed on May 20	d6b0f49

E. Lampiran Source Code Utama

Main.py

```

1 # main.py
2 # Proyek Akhir Algoritma & Struktur Data - Kelompok 1
3 # Ketua: Muhammad Ramdhan Maulana
4
5 import os
6 import time
7 from engine import PriorityQueue
8 from storage import load_data, save_data
9 from interface import header_klinik, display_menu, animasi_panggil, Color
10
11
12 def clear_screen():
13     os.system('cls' if os.name == 'nt' else 'clear')
14
15 # -----
16 # SUBMENU: Fitur Pencarian & Pengurutan Pasien (Menu 4)
17 # -----
18 def menu_filter_search(antrian):
19     while True:
20         clear_screen()
21         print(f"{Color.CYAN}{'-' * 60}")
22         print(f"{Color.RESET}{Color.BOLD}PENCARIAN, PENGURUTAN & FILTER ANTRIAN{Color.RESET}".center(60))
23         print(f"{Color.RESET}Fitur Analisis dan Filtrasi Data Pasien{Color.RESET}".center(60))
24         print(f"{Color.CYAN}{'-' * 60}{Color.RESET}")
25
26         print(f" {Color.CYAN}— Opsi Pengurutan (Sorting) —{Color.RESET}")
27         print(f" [{Color.GREEN}1{Color.RESET}] Urut Alfabetis      (A-Z)" )
28         print(f" [{Color.GREEN}2{Color.RESET}] Urut Urgensi      (Darurat + Normal)" )
29         print(f" [{Color.GREEN}3{Color.RESET}] Urut Gabungan      (Darurat A-Z + Normal A-Z)" )
30         print()
31         print(f" {Color.CYAN}— Opsi Pencarian (Searching) —{Color.RESET}")
32         print(f" [{Color.GREEN}4{Color.RESET}] Cari Pasien      (Nama / ID / Tanggal)" )
33         print()
34         print(f" {Color.CYAN}— Opsi Penyaringan (Filtering) —{Color.RESET}")
35         print(f" [{Color.GREEN}5{Color.RESET}] Tampilkan Khusus Pasien Darurat" )
36         print(f" [{Color.GREEN}6{Color.RESET}] Tampilkan Khusus Pasien Normal" )
37         print()
38         print(f" [{Color.RED}0{Color.RESET}] Kembali ke Menu Utama")
39         print(f"{Color.CYAN}{'-' * 60}{Color.RESET}")
40
41         pilihan = input(f" Pilih menu [0-6]: {Color.BOLD}")
42         print(Color.RESET, end="")
43
44         if pilihan == '0':
45             break
46
47         elif pilihan == '1':
48             clear_screen()
49             print(f"\n{Color.YELLOW}=== ANTRIAN ALFABETIS (A-Z) ==={Color.RESET}")
50             antrian.display_sorted("nama")
51             input(f"\n{Color.CYAN}Tekan Enter untuk kembali...{Color.RESET}")
52
53         elif pilihan == '2':
54             clear_screen()
55             print(f"\n{Color.YELLOW}=== ANTRIAN BERDASARKAN URGENSI ==={Color.RESET}")
56             antrian.display_sorted("kategori")
57             input(f"\n{Color.CYAN}Tekan Enter untuk kembali...{Color.RESET}")
58

```

```

64
65     elif pilihan == '4':
66         clear_screen()
67         print(f"\n{Color.CYAN}=== PENCARIAN DATA PASIEN ==={Color.RESET}\n")
68         keyword = input(f" Masukkan Nama / ID / Tanggal Pasien: {Color.BOLD}").strip()
69         print(Color.RESET, end="")
70         if keyword:
71             antrian.search_pasien(keyword)
72         else:
73             print(f"\n{Color.RED}❌ Keyword pencarian tidak boleh kosong!{Color.RESET}")
74             input(f"\n{Color.CYAN}Tekan Enter untuk kembali...{Color.RESET}")
75
76     # --- FITUR BARU: FILTER KATEGORI ---
77     elif pilihan == '5':
78         clear_screen()
79         antrian.display_by_kategori("Darurat")
80         input(f"\n{Color.CYAN}Tekan Enter untuk kembali...{Color.RESET}")
81
82     elif pilihan == '6':
83         clear_screen()
84         antrian.display_by_kategori("Normal")
85         input(f"\n{Color.CYAN}Tekan Enter untuk kembali...{Color.RESET}")
86
87     #
88     else:
89         print(f"\n{Color.RED}❌ Pilihan tidak valid! Masukkan angka antara 0 sampai 6.{Color.RESET}")
90         input(f"{Color.YELLOW} Tekan [Enter] untuk mencoba lagi...{Color.RESET}")
91
92 # -----
93 # ALUR UTAMA PROGRAM
94 # -----
95 def main():
96     antrian = PriorityQueue()
97
98     # Load data lama dari CSV jika ada
99     load_data(antrian)
100
101     # Load fungsi login
102     from interface import login_admin
103     while True:
104         clear_screen()
105         # Jika fungsi login mengembalikan True, keluar dari loop login dan masuk ke aplikasi
106         if login_admin():
107             break
108
109     while True:
110         clear_screen()
111         header_klinik()
112
113         # Dashboard Statistik Terstandarisasi Warna
114         print(f" STATS: [Menunggu: {Color.RED}{antrian.size()}{Color.RESET}] | [Pasien dilayani: {Color.GREEN}{antrian.get_log_count()}{Color.RESET}]"
115               f"{Color.CYAN}{'-' * 60}{Color.RESET}")
116
117         display_menu()
118         pilihan = input(f"Pilih menu [0-4]: {Color.BOLD}").strip()
119         print(Color.RESET, end="") # Reset warna input utama
120

```

```

121
122 if pilihan == '1':
123     # --- LOOPING MULTI-INPUT REGISTRASI PASIEN ---
124     while True:
125         clear_screen()
126         print(f"\n{Color.CYAN}=== REGISTRASI PASIEN BARU ==={Color.RESET}\n")
127         nama = input(f" Nama Pasien : {Color.BOLD}").strip()
128         print(Color.RESET, end="")
129         if not nama:
130             print(f"\n{Color.RED}❌ Nama tidak boleh kosong!{Color.RESET}")
131             input(f"\n{Color.CYAN} Tekan Enter untuk mencoba lagi...{Color.RESET}")
132             continue
133
134         # 🐛 BUG HANDLING: VERIFIKASI LOCK Y/N
135         print(f"\n {Color.YELLOW}[VERIFIKASI ADMIN]{Color.RESET}")
136         print(f" Apakah nama {Color.BOLD}'{nama}'{Color.RESET} sudah benar dan sesuai KTP/BPJS?")
137
138         while True:
139             verif = input(f" Konfirmasi [Y/N]: {Color.BOLD}").strip().lower()
140             print(Color.RESET, end="")
141             if verif in ['y', 'n']:
142                 break
143             print(f" {Color.RED}❌ Masukan salah. Ketik Y jika benar, atau N untuk membatalkan.{Color.RESET}")
144
145         if verif == 'n':
146             print(f"\n{Color.RED}❌ Pendaftaran dibatalkan. Data tidak dimasukkan ke antrian.{Color.RESET}")
147             input(f"{Color.CYAN} Tekan Enter untuk mengulang...{Color.RESET}")
148             continue # Mengulang loop registrasi dari awal (input nama lagi)
149
150         print(f"\n {Color.YELLOW}Kategori Kondisi Pasien:{Color.RESET}")
151         print(f" [{Color.GREEN}1{Color.RESET}] Normal")
152         print(f" [{Color.GREEN}2{Color.RESET}] Darurat (Prioritas)")
153
154         while True:
155             kat_input = input(f" Pilih [1/2] : {Color.BOLD}").strip()
156             print(Color.RESET, end="")
157             if kat_input in ['1', '2']:
158                 break
159             print(f" {Color.RED}❌ Pilihan salah. Ketik 1 untuk Normal atau 2 untuk Darurat.{Color.RESET}")
160
161         kategori = "Darurat" if kat_input == '2' else "Normal"
162         new_id = antrian.enqueue(nama, kategori)
163         print(f"\n{Color.GREEN}🟢 Pasien {Color.BOLD}{new_id}{Color.RESET}{Color.GREEN} berhasil terdaftar!{Color.RESET}")
164
165     # 📌 PERTANYAAN INPUT ULANG (MULTI-INPUT LOCK) ---
166     print(f"\n{Color.CYAN}{'-' * 45}{Color.RESET}")
167     print(f" Apakah ingin mendaftarkan pasien lain?")
168     while True:
169         ulang = input(f" Input lagi? [Y/N]: {Color.BOLD}").strip().lower()
170         print(Color.RESET, end="")
171         if ulang in ['y', 'n']:
172             break
173         print(f" {Color.RED}❌ Masukan salah. Ketik Y untuk input lagi, atau N untuk kembali ke menu utama.{Color.RESET}")
174
175     if ulang == 'n':
176         break # Keluar dari loop registrasi dan kembali ke Menu Utama
177
178 # -----
179
180 elif pilihan == '2':
181     clear_screen()
182     pasien_dipanggil = antrian.dequeue()
183     if pasien_dipanggil:
184         animasi_panggil(pasien_dipanggil)
185     else:
186         print(f"\n {Color.RED}Antrian kosong! Tidak ada pasien di ruang tunggu.{Color.RESET}")
187         input(f"\n{Color.CYAN} Tekan Enter...{Color.RESET}")
188
189 elif pilihan == '3':
190     clear_screen()

```

```

178
179     elif pilihan == '2':
180         clear_screen()
181         pasien_dipanggil = antrian.dequeue()
182         if pasien_dipanggil:
183             animasi_panggil(pasien_dipanggil)
184         else:
185             print(f"\n (Color.RED)Antrian kosong! Tidak ada pasien di ruang tunggu.(Color.RESET)")
186             input(f"\n(Color.CYAN) Tekan Enter...(Color.RESET)")
187
188     elif pilihan == '3':
189         clear_screen()
190         print(f"\n(Color.CYAN)DAFTAR ANTRIAN AKTIF(Color.RESET)".center(52))
191         antrian.display_all()
192         input(f"\n(Color.CYAN)Tekan Enter untuk kembali...(Color.RESET)")
193
194     elif pilihan == '4':
195         menu_filter_search(antrian)
196
197     elif pilihan == '0':
198         save_data(antrian)
199         print(f"\n(Color.GREEN)Sampai jumpa kembali!(Color.RESET)\n")
200         break
201
202     else:
203         # Bug Handling Utama: Menghentikan layar untuk memberikan feedback error
204         print(f"\n(Color.RED)❌ Pilihan menu tidak tersedia! Masukkan angka antara 0 sampai 4.(Color.RESET)")
205         input(f"\n(Color.YELLOW) Tekan [Enter] untuk kembali ke menu utama...(Color.RESET)")
206
207 if __name__ == "__main__":
208     main()

```

Interface.py

```

1 # Interface.py
2 # Modul Tampilan / UI - Kelompok 1
3
4 import time
5 import os
6 import msvcrt
7 import sys
8
9 # Definisi Warna ANSI
10 class Color:
11     CYAN = '\033[96m'
12     GREEN = '\033[92m'
13     YELLOW = '\033[93m'
14     RED = '\033[91m'
15     BOLD = '\033[1m'
16     UNDERLINE = '\033[4m'
17     RESET = '\033[0m'
18     BLUE = '\033[94m'
19
20 def header_klinik():
21     """Menampilkan banner utama aplikasi dengan tulisan putih dan pembatas cyan."""
22     print(f"{Color.CYAN}{ '-' * 60}")
23     print(f"{Color.RESET}SISTEM ANTRIAN KLINIK{Color.RESET}".center(60))
24     print(f"{Color.RESET}Pelayanan Kesehatan Cepat & Responsif{Color.RESET}".center(60))
25     print(f"{Color.CYAN}{ '-' * 60}{Color.RESET}")
26
27 def display_menu():
28     """Menampilkan daftar pilihan menu dengan format yang rapi."""
29     menu_items = [
30         "Registrasi Pasien Baru",
31         "Panggil Pasien Berikutnya",
32         "Lihat Antrian Aktif (Fix Order)",
33         "Pencarian & Pengurutan Pasien",
34         "Keluar & Simpan Data"
35     ]
36
37     '''Memberikan warna untuk nomor pada menu.'''
38     for i, item in enumerate(menu_items):
39         # Jika item terakhir (Keluar & Simpan Data), set nomornya menjadi 0
40         if i == len(menu_items) - 1:
41             num_str = f"{{Color.RED}}0{{Color.RESET}}}"
42         else:
43             num_str = f"{{Color.GREEN}}{i+1}{{Color.RESET}}}"
44
45         print(f"{{num_str}} {{item}}")
46     print(f"{Color.CYAN}{ '-' * 60}{Color.RESET}")
47
48 def loading_spinner(duration=1, message="Memproses"):
49
50     """Animasi spinner sederhana."""
51     symbols = ['|', '/', '-', '\\']
52     end_time = time.time() + duration
53     i = 0
54     while time.time() < end_time:

```

```

55     print(f"\r{Color.YELLOW} {symbols[i % 4]} {message}...(Color.RESET)", end="", flush=True)
56     time.sleep(0.1)
57     i += 1
58     print("\r" + " " * (len(message) + 10) + "\r", end="")
59
60 def animasi_panggil(pasien):
61     """Memberikan efek visual saat memanggil pasien."""
62     loading_spinner(1.5, "MENGHUBUNGKAN KE SPEAKER")
63
64     '''Menentukan warna berdasarkan kategori pasien'''
65     border_color = Color.RED if pasien.kategori == "Darurat" else Color.GREEN
66     text_style = Color.BOLD + Color.RED if pasien.kategori == "Darurat" else Color.BOLD + Color.GREEN
67     inner_width = 58
68
69     print("\n" + border_color + "+" + "-" * inner_width + "+")
70     print("|" + "PASIEN HARAP MENUJU RUANG DOKTER".center(inner_width) + "|")
71     print("|" + "-" * inner_width + "|" + Color.RESET)
72
73     # Baris data dengan padding dinamis agar bingkai kanan (|) tetap sejajar
74     line_id = f" ID PASIEN : {pasien.id_pasien}"
75     print(f"{border_color}| {Color.BOLD}ID PASIEN :{Color.RESET} {pasien.id_pasien}" + " " * (inner_width - len(line_id)) + f"{border_color}|")
76
77     line_nama = f" NAMA : {pasien.nama.upper()}"
78     print(f"{border_color}| {Color.BOLD}NAMA :{Color.RESET} {pasien.nama.upper()}" + " " * (inner_width - len(line_nama)) + f"{border_color}|")
79
80     line_status = f" STATUS : {pasien.kategori.upper()}"
81     print(f"{border_color}| {Color.BOLD}STATUS :{Color.RESET} {text_style}{pasien.kategori.upper()}{Color.RESET}" +
82           " " * (inner_width - len(line_status)) + f"{border_color}|")
83
84     if pasien.kategori == "Darurat":
85         msg = " ⚠ PERINGATAN: SEGERA BERIKAN TINDAKAN MEDISI!"
86         print(f"{border_color}|{Color.RED}{msg}{Color.RESET}" + " " * (inner_width - len(msg)) + f"{border_color}|")
87     else:
88         msg = " i CATATAN: Mohon menunggu giliran dengan tertib."
89         print(f"{border_color}|{Color.BLUE}{msg}{Color.RESET}" + " " * (inner_width - len(msg)) + f"{border_color}|")
90
91     print(border_color + "+" + "-" * inner_width + "+" + Color.RESET)
92
93     input(f"\n{Color.YELLOW}Tekan [Enter] setelah pasien selesai dilayani...(Color.RESET)")
94     print(f"\n{Color.GREEN}✅ Pasien telah selesai. Data dipindahkan ke Riwayat.{Color.RESET}")
95     time.sleep(1)
96
97
98 def tabel_kosong():
99     """Tampilan jika data tidak ditemukan."""
100     print(f"\n{Color.RED}{'_' * 60}")
101     print("| DATA TIDAK DITEMUKAN / ANTRIAN KOSONG |")
102     print(f"{'_' * 60}{Color.RESET}")
103
104 def login_admin():
105     """Fungsi login admin dengan sensor bintang (*) khusus Windows."""
106     # Pengaturan akun admin default klinik

```

```

107     USERNAME_BENAR = "admin"
108     PASSWORD_BENAR = "admin123"
109
110     print(f"{Color.CYAN}{'-' * 60}")
111     print(f"{Color.RESET}{Color.BOLD}OTENTIKASI LOGIN SISTEM KLINIK{Color.RESET}".center(60))
112     print(f"{Color.CYAN}{'-' * 60}{Color.RESET}\n")
113
114     username = input(f" Username Admin : {Color.BOLD}")
115     print(Color.RESET, end="")
116
117     print(" Password Admin : ", end="", flush=True)
118
119     password = ""
120     while True:
121         # Membaca karakter yang diketik secara langsung tanpa memunculkannya di layar
122         ch = msvcrt.getch()
123
124         # Jika user menekan Enter (\r atau \n)
125         if ch in [b'\r', b'\n']:
126             print() # Pindah baris baru setelah menekan enter
127             break
128
129         # Jika user menekan Backspace (\x08) untuk menghapus
130         elif ch == b'\x08':
131             if len(password) > 0:
132                 password = password[:-1]
133                 # Mundurkan kursor terminal, hapus karakter dengan spasi, lalu mundurkan lagi
134                 sys.stdout.write("\b \b")
135                 sys.stdout.flush()
136
137         # Jika karakter normal diketik
138         else:
139             try:
140                 char_decode = ch.decode('utf-8')
141                 password += char_decode
142                 # Tampilkan karakter bintang sebagai sensor di layar terminal
143                 sys.stdout.write('*')
144                 sys.stdout.flush()
145             except:
146                 pass # Mengabaikan karakter aneh / tombol fungsi (seperti F1-F12 atau arrow)
147
148     # Validasi Kecocokan Kredensial Akun
149     if username == USERNAME_BENAR and password == PASSWORD_BENAR:
150         print(f"\n{Color.GREEN}✅ Verifikasi Berhasil! Selamat datang, Admin.{Color.RESET}")
151         import time
152         time.sleep(1.2)
153         return True
154     else:
155         print(f"\n{Color.RED}❌ Username atau Password salah! Akses ditolak.{Color.RESET}")
156         input(f"{Color.YELLOW} Tekan Enter untuk mencoba lagi...(Color.RESET)")
157         return False

```

engine.py

```
1 # engine.py
2 # Model Logika Struktur Data - Kelompok 1
3
4 from interface import Color # Import Color untuk estetika
5 from datetime import datetime, timedelta, timezone
6 from interface import Color
7 from storage import save_data
8
9 class Node:
10     def __init__(self, id_pasien, nama, kategori, waktu):
11         self.id_pasien = id_pasien
12         self.nama = nama
13         self.kategori = kategori # "Normal" atau "Darurat"
14         self.waktu = waktu # Format: "YYYY-MM-DD HH:MM:SS"
15         self.next = None
16
17 class PriorityQueue:
18     def __init__(self):
19         self.head = None
20         self.tail = None
21         self.last_emergency = None # Pointer ke pasien darurat terakhir (untuk FIFO sesama darurat)
22         self.last_id_num = 0 # Counter untuk ID unik
23         self.size = 0 # Counter efisien untuk jumlah antrian
24         self.logs = [] # Simpan riwayat (Stack-like)
25
26     def is_empty(self):
27         return self.head is None
28
29     def size(self):
30         """Mengembalikan jumlah antrian secara instan O(1)."""
31         return self._size
32
33     def set_last_id(self, id_str):
34         """Memastikan ID baru tidak bentrok dengan ID lama di CSV."""
35         try:
36             num = int(id_str.replace("P", ""))
37             if num > self.last_id_num:
38                 self.last_id_num = num
39         except:
40             pass
41
42     def enqueue(self, nama, kategori, manual_id=None, manual_waktu=None):
43         """
44         Jika manual_id/waktu ada (saat load data), gunakan itu.
45         Jika tidak (pasien baru), buat ID baru secara otomatis.
46         """
47         if manual_id:
48             new_id = manual_id
49             self.set_last_id(manual_id)
50         else:
51             self.last_id_num += 1
52             new_id = f"P{self.last_id_num:03d}"
53
54         # Set waktu ke GMT+7 jika tidak ada manual_waktu
55         if manual_waktu:
56             waktu_final = manual_waktu
57         else:
58             tz_get7 = timezone(timedelta(hours=7))
59             waktu_final = datetime.now(tz_get7).strftime("%Y-%m-%d %H:%M:%S")
60
61         new_node = Node(new_id, nama, kategori, waktu_final)
62         self._size += 1
63
64         # LOGIKA PRIORITY
65         if self.is_empty():
66             self.head = self.tail = new_node
67             if kategori == "Darurat":
68                 self.last_emergency = new_node
```

```

69     elif kategori == "Darurat":
70         # Darurat dimasukkan setelah pasien darurat terakhir, sebelum pasien normal (FIFO Priority)
71         if self.last_emergency:
72             new_node.next = self.last_emergency.next
73             self.last_emergency.next = new_node
74             if self.last_emergency == self.tail:
75                 self.tail = new_node
76             self.last_emergency = new_node
77         else:
78             # Pasien darurat pertama di tengah antrian normal
79             new_node.next = self.head
80             self.head = new_node
81             self.last_emergency = new_node
82         else:
83             # Normal ke belakang (Tail)
84             self.tail.next = new_node
85             self.tail = new_node
86
87     save_data(self)
88     return new_id
89
90     def dequeue(self):
91         if self.is_empty():
92             return None
93
94         temp = self.head
95         self.head = self.head.next
96         self._size -= 1
97
98         if self.head is None:
99             self.tail = None
100             self.last_emergency = None
101         elif temp == self.last_emergency:
102             self.last_emergency = None
103
104         # Masukkan ke Log (Gunakan append agar O(1))
105         self.logs.append(temp)
106         save_data(self)
107         return temp
108
109     # -----
110     # HELPER: ambil semua node dari linked list
111     # -----
112     def _to_list(self):
113         """Mengonversi linked list ke Python list (hanya untuk keperluan tampilan/sort)."""
114         result = []
115         curr = self.head
116         while curr:
117             result.append(curr)
118             curr = curr.next
119         return result
120
121     # -----
122     # DISPLAY (tampilan asli / urutan antrian)
123     # -----
124     def _print_table(self, nodes, judul="ANTRIAN AKTIF"):
125         """Mencetak tabel dari list node dengan pewarnaan terstandarisasi."""
126         print("\n" + f"({Color.CYAN})=" * 73)
127         print(f"({Color.RESET})(judul)({Color.RESET}).center(65))
128         print(f"({Color.CYAN})=" * 73)
129         print(f"| {'No':<4} | {'ID':<6} | {'NAMA PASIEN':<20} | {'STATUS':<10} | {'WAKTU DAFTAR'}")
130         print(f"|" * 73 + f"({Color.RESET})")
131
132         for i, node in enumerate(nodes, start=1):
133             if node.kategori == "Darurat":
134                 kat_warna = f"({Color.RED})(node.kategori:<10)({Color.RESET})"
135             else:

```

```

137     kat_warna = f"({Color.GREEN}{node.kategori:<10}{Color.RESET})"
138
139     print(f" | {i:<4} | {node.id_pasien:<6} | {node.nama:<20} | {kat_warna} | {node.waktu}")
140
141     print(f"({Color.CYAN})=" * 73 + f"({Color.RESET})")
142     print(f" Total: {Color.BOLD}{len(nodes)}{Color.RESET} pasien\n")
143
144     def display_all(self):
145         if self.is_empty():
146             print(f"\n({Color.RED})--- Antrian Kosong ---{Color.RESET}")
147             return
148
149         # Header Tabel menggunakan warna Cyan agar konsisten dengan menu utama
150         print("\n" + f"({Color.CYAN})=" * 52 + f"({Color.RESET})")
151         print(f" | {i'D':<6} | {i'NAMA PASIEN':<20} | {i'KATEGORI':<12} | ")
152         print(f"({Color.CYAN})=" * 52 + f"({Color.RESET})")
153
154         curr = self.head
155         while curr:
156             # Berikan warna pembeda pada kolom kategori secara dinamis
157             if curr.kategori == "Darurat":
158                 kat_warna = f"({Color.RED}){curr.kategori:<12}{Color.RESET}"
159             else:
160                 kat_warna = f"({Color.GREEN}){curr.kategori:<12}{Color.RESET}"
161
162             # Cetak baris data pasien
163             print(f"({Color.CYAN}){Color.RESET} {curr.id_pasien:<6} {Color.CYAN}{Color.RESET} {curr.nama:<20} {Color.CYAN}{Color.RESET} {kat_warna} {Color.CYAN}{Color.RESET}")
164             curr = curr.next
165
166         print(f"({Color.CYAN})=" * 52 + f"({Color.RESET})")
167
168     # =====
169     # SORT - Hanya untuk tampilan, TIDAK mengubah antrian asli
170     # =====
171     def display_sorted(self, mode):
172         """
173         Menampilkan antrian dalam urutan tertentu (TIDAK mengubah antrian asli).
174         mode:
175         'nama' / 'alpha' -> Urut A-Z berdasarkan nama pasien
176         'kategori' / 'urgency' -> Darurat semua dulu, lalu Normal (urutan kedatangan dipertahankan)
177         'gabungan' / 'combined' -> Darurat dulu (A-Z), lalu Normal (A-Z)
178         """
179         if self.is_empty():
180             print(f"\n({Color.YELLOW})")
181             print(f" | {Color.BOLD}ANTRIAN KOSONG SAAT INI{Color.RESET}{Color.YELLOW} | ")
182             print(f" | {Color.RESET} ")
183             return
184
185         # Ambil semua node dari linked list ke dalam list penolong (Array lokal)
186         nodes = self._to_list()
187
188         if mode in ['nama', 'alpha']:
189             # Merge Sort berdasarkan nama (A-Z)
190             sorted_nodes = self._merge_sort_alpha(nodes)
191             judul = "ANTRIAN - URUT ALFABETIS (A-Z)"
192
193         elif mode in ['kategori', 'urgency']:
194             # Pisahkan Darurat & Normal, pertahankan urutan kedatangan masing-masing (FIFO Priority)
195             darurat = [n for n in nodes if n.kategori == "Darurat"]
196             normal = [n for n in nodes if n.kategori == "Normal"]
197             sorted_nodes = darurat + normal
198             judul = "ANTRIAN - URUT URGENSI (Darurat + Normal)"
199
200         elif mode in ['gabungan', 'combined']:
201             # Merge Sort - Darurat A-Z dulu, lalu Normal A-Z
202             darurat = self._merge_sort_alpha([n for n in nodes if n.kategori == "Darurat"])
203             normal = self._merge_sort_alpha([n for n in nodes if n.kategori == "Normal"])
204             sorted_nodes = darurat + normal
205             judul = "ANTRIAN - URGENSI + ALFABETIS (Darurat A-Z + Normal A-Z)"
206
207         else:
208             print(f"({Color.RED})[ERROR] Mode sort tidak dikenali.{Color.RESET}")
209             return
210
211         # Panggil fungsi pembantu untuk menampilkan array yang sudah terurut
212         self._print_table(sorted_nodes, judul=judul)
213
214     def _merge_sort_alpha(self, nodes):
215         """Merge sort A-Z berdasarkan nama (case-insensitive). O(n log n)"""
216         if len(nodes) <= 1:
217             return nodes[:]
218
219         def merge(left, right):
220             """Menggabungkan dua array yang sudah terurut."""
221             result = []
222             i = j = 0
223             while i < len(left) and j < len(right):
224                 if left[i].nama.lower() <= right[j].nama.lower():
225                     result.append(left[i])
226                     i += 1
227                 else:
228                     result.append(right[j])
229                     j += 1
230             result.extend(left[i:])
231             result.extend(right[j:])
232             return result
233
234         def merge_sort(arr):
235             """Divide and conquer - membagi array hingga ukuran 1, lalu menggabungkannya."""
236             if len(arr) <= 1:
237                 return arr
238             mid = len(arr) // 2
239             left = merge_sort(arr[:mid])
240             right = merge_sort(arr[mid:])
241             return merge(left, right)
242
243         return merge_sort(nodes[:])
244
245     # =====
246     # SEARCH - partial match, semua hasil ditampilkan
247     # =====
248     def search_pasien(self, keyword):
249         """
250         Mencari pasien berdasarkan nama (partial), ID (exact), atau waktu (partial).
251         Contoh: keyword "Michael" - menemukan "Michael Flow" DAN "Michael Burt".
252         """
253         keyword_lower = keyword.lower().strip()
254         curr = self.head
255         hasil = []
256
257         while curr:
258             cocok_nama = keyword_lower in curr.nama.lower()
259             cocok_id = keyword_upper() == curr.id_pasien
260             cocok_waktu = keyword in curr.waktu
261             if cocok_nama or cocok_id or cocok_waktu:
262                 hasil.append(curr)
263             curr = curr.next
264
265         print("\n" + "-" * 60)
266         if hasil:
267             print(f" Hasil pencarian untuk: '{keyword}'\n ({len(hasil)} ditemukan)")
268             print("-" * 60)
269             print(f" | {i'No':<4} | {i'D':<7} | {i'NAMA PASIEN':<18} | {i'STATUS':<10} | {i'WAKTU DAFTAR':<10} | ")
270             print(f" | {i'-' * 60} ")
271             for i, node in enumerate(hasil, start=1):
272                 status_mark = "●" if node.kategori == "Darurat" else "●"

```



```

273         print(f" {i<4} {node.id_pasien:<7} {node.nama:<18} {status_mark} {node.kategori:<8} {node.waktu}")
274     print("\n" * 60)
275 else:
276     print(f" Tidak ada pasien yang cocok dengan \"{keyword}\".")
277     print("\n" * 60)
278
279 # =====
280 # LOG & RIWAYAT
281 # =====
282 def show_logs(self):
283     if not self.logs:
284         print(f"\n{Color.YELLOW} | ")
285         print(f" | {Color.BOLD}BELUM ADA PASIEN DILAYANI{Color.RESET}{Color.YELLOW} | ")
286         print(f" | {Color.RESET} ")
287     return
288
289     print(f"\n{Color.BLUE} | ")
290     print(f" | {Color.BOLD}RIWAYAT PELAYANAN (LOG STACK){Color.RESET}{Color.BLUE} ", justify="right", width=60)
291     print(f" | {Color.RESET} ")
292     for p in reversed(self.logs): # Tampilkan dari yang terbaru
293         print(f" | {Color.BLUE} | [{p.waktu}] [{p.id_pasien}] - {p.nama} {Color.GREEN}(SELESAI){Color.RESET}{Color.BLUE} ", justify="right", width=75)
294         print(f" | {Color.BLUE} | {Color.RESET} ")
295
296 def search_pasien(self, keyword):
297     curr = self.head
298     found = False
299     print(f"\n{Color.YELLOW} Mencari pasien dengan keyword '{keyword}'...{Color.RESET}")
300     while curr:
301         # Sekarang bisa mencari berdasarkan Nama, ID, atau bagian dari Waktu (misal: "2023-10")
302         if (keyword.lower() in curr.nama.lower() or
303             keyword.upper() == curr.id_pasien or
304             keyword in curr.waktu):
305             print(f"{Color.GREEN} ■ DITEMUKAN: {Color.BOLD}{curr.id_pasien}{Color.RESET} - {curr.nama} [{curr.kategori}] (Daftar: {curr.waktu}){Color.RESET}")
306             found = True
307             curr = curr.next
308         if not found:
309             print(f"{Color.RED} ✗ Pasien '{keyword}' tidak ditemukan.{Color.RESET}")
310
311 def get_log_count(self):
312     """Mengembalikan jumlah pasien yang sudah dilayani."""
313     return len(self.logs)
314
315 def display_by_kategori(self, target_kategori):
316     """
317     Menampilkan daftar antrian yang difilter berdasarkan kategori tertentu.
318     target_kategori: "Darurat" atau "Normal"
319     """
320     if self.is_empty():
321         print(f"\n{Color.YELLOW} | ")
322         print(f" | {Color.BOLD}ANTRIAN KOSONG SAAT INI{Color.RESET}{Color.YELLOW} | ")
323         print(f" | {Color.RESET} ")
324     return
325
326     # Ambil semua node linked list ke dalam list penolong
327     nodes = self._to_list()
328     # Filter hanya yang kategorinya cocok
329     filtered_nodes = [n for n in nodes if n.kategori == target_kategori]
330
331     if not filtered_nodes:
332         print(f"\n{Color.RED} ✗ Tidak ada pasien dengan kategori '{target_kategori}' di dalam antrian.{Color.RESET}")
333     return
334
335     judul = f"ANTRIAN AKTIF - KHUSUS KATEGORI {target_kategori.upper()}"
336
337     # Manfaatkan fungsi cetak tabel yang sudah kita perbaiki sebelumnya
338     self._print_table(filtered_nodes, judul=judul)

```

storage.py

```
1  # storage.py
2  # Modul File Handling (CSV) - Kelompok 1
3
4  import csv
5  import os
6
7  # Nama file penyimpanan
8  FILE_ANTRIAN = 'antrian.csv'
9  FILE_LOG = 'log_pelayanan.csv'
10
11 def load_data(queue_obj):
12     """Membaca data dari CSV ke dalam struktur data Linked List saat start."""
13     # Load Antrian Aktif
14     if os.path.exists(FILE_ANTRIAN):
15         try:
16             with open(FILE_ANTRIAN, mode='r', newline='') as file:
17                 reader = csv.DictReader(file)
18                 for row in reader:
19                     # Kita masukkan kembali ke queue
20                     # Sertakan manual_id & manual_waktu agar data konsisten
21                     queue_obj.enqueue(
22                         row['nama'],
23                         row['kategori'],
24                         manual_id=row['id'],
25                         manual_waktu=row.get('waktu', '-')
26                     )
27             print(f"[STORAGE] Berhasil memuat antrian dari {FILE_ANTRIAN}")
28         except Exception as e:
29             print(f"[ERROR] Gagal membaca file antrian: {e}")
30
31     # Load Log Pelayanan (Opsional untuk Stack)
32     # Catatan: Karena logs di engine.py adalah list, kita tinggal append
33     if os.path.exists(FILE_LOG):
34         try:
35             with open(FILE_LOG, mode='r', newline='') as file:
36                 reader = csv.DictReader(file)
37                 for row in reader:
38                     # Membuat objek dummy/Node untuk log
39                     from engine import Node
40                     node_log = Node(
41                         row['id'],
42                         row['nama'],
43                         row['kategori'],
44                         row.get('waktu', '-')
45                     )
46                     queue_obj.logs.append(node_log)
47         except Exception as e:
48             print(f"[ERROR] Gagal membaca log: {e}")
49
```

```

50 def save_data(queue_obj):
51     """Menyimpan data dari Linked List ke dalam CSV saat exit."""
52     # 1. Simpan Antrian yang masih menunggu
53     try:
54         with open(FILE_ANTRIAN, mode='w', newline='') as file:
55             fieldnames = ['id', 'nama', 'kategori', 'waktu']
56             writer = csv.DictWriter(file, fieldnames=fieldnames)
57             writer.writeheader()
58
59             curr = queue_obj.head
60             while curr:
61                 writer.writerow({
62                     'id': curr.id_pasien,
63                     'nama': curr.nama,
64                     'kategori': curr.kategori,
65                     'waktu': curr.waktu
66                 })
67                 curr = curr.next
68     except Exception as e:
69         print(f"[ERROR] Gagal menyimpan antrian: {e}")
70
71     # 2. Simpan Log Pelayanan (History)
72     try:
73         with open(FILE_LOG, mode='w', newline='') as file:
74             fieldnames = ['id', 'nama', 'kategori', 'waktu']
75             writer = csv.DictWriter(file, fieldnames=fieldnames)
76             writer.writeheader()
77
78             for p in queue_obj.logs:
79                 writer.writerow({
80                     'id': p.id_pasien,
81                     'nama': p.nama,
82                     'kategori': p.kategori,
83                     'waktu': p.waktu
84                 })
85     except Exception as e:
86         print(f"[ERROR] Gagal menyimpan log: {e}")

```